

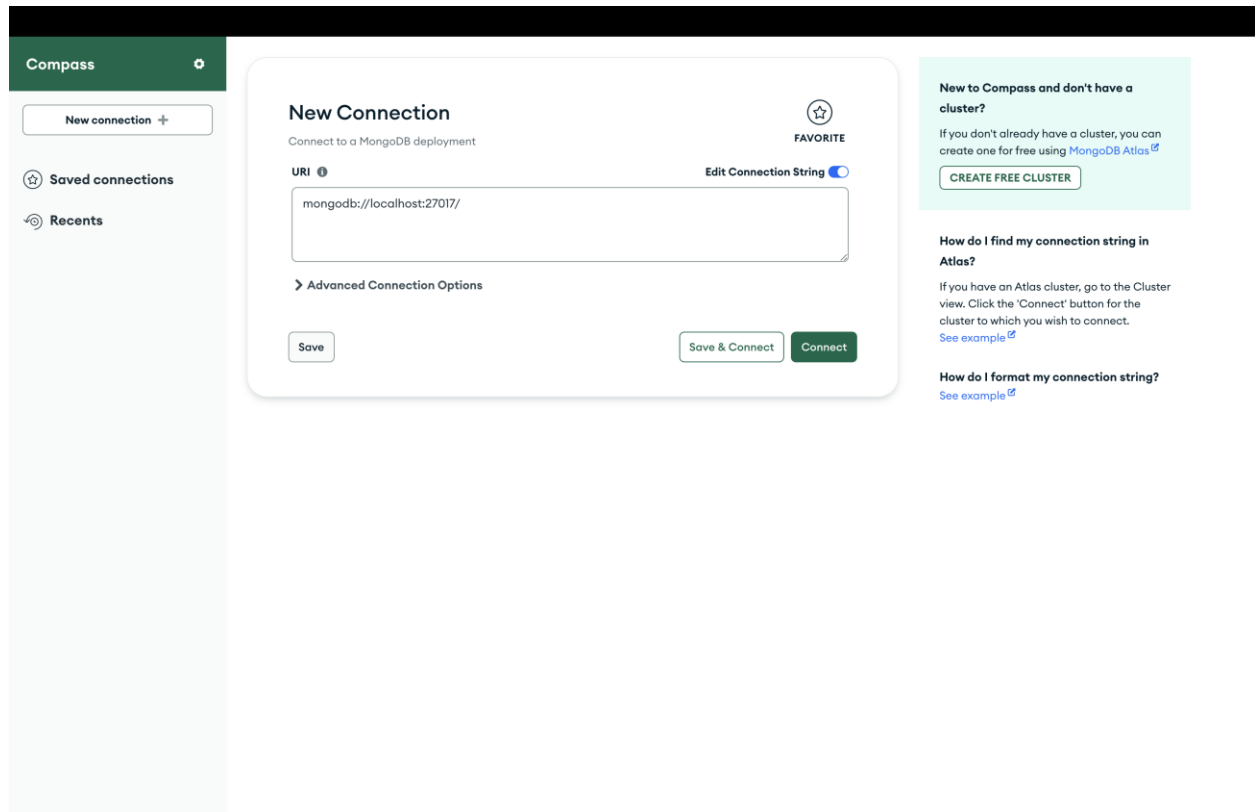
Jeevan Kumar Banoth – 02105145

CIS 552: Database Design

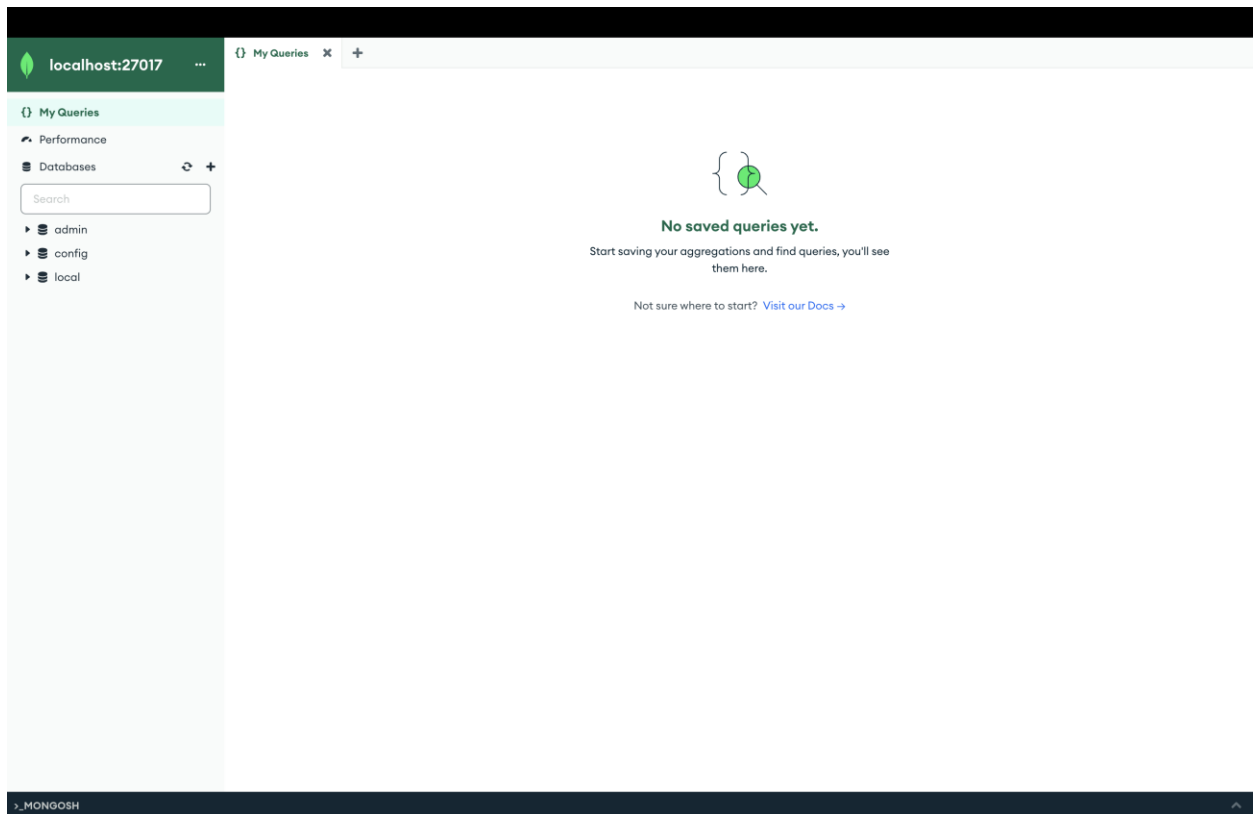
Homework – 7:

Task 1: Starting a New Database and Collection

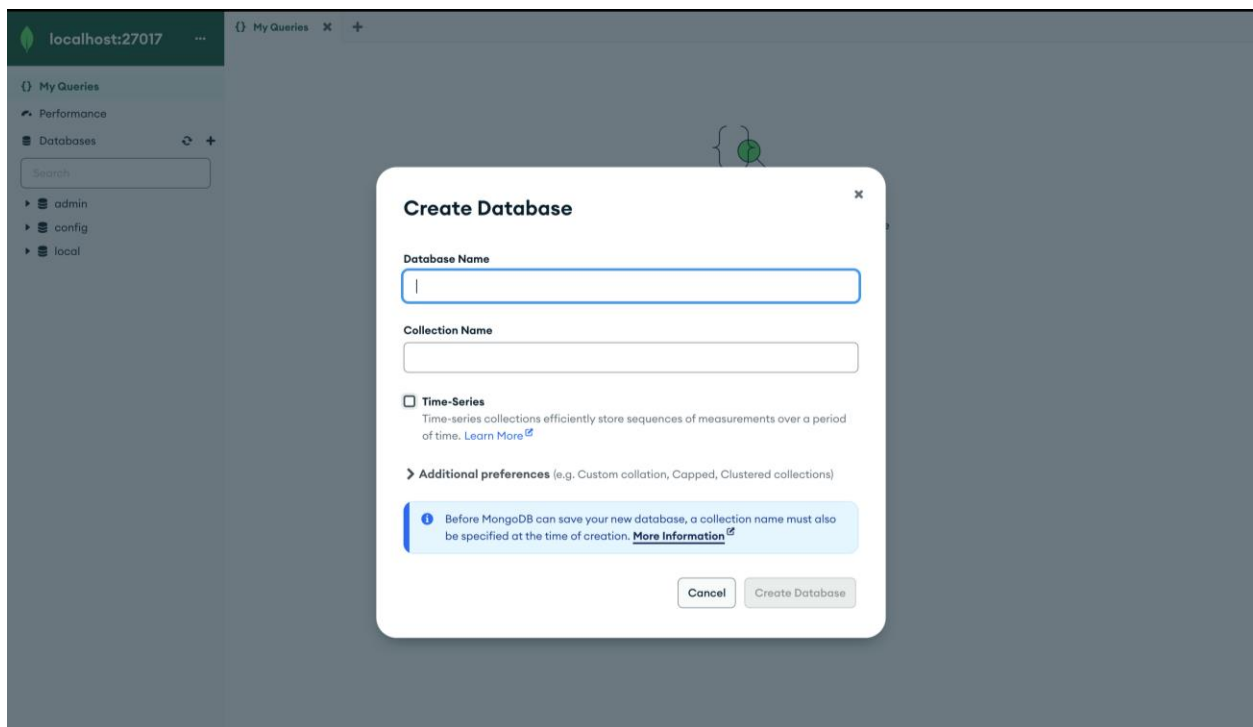
To connect to the MongoDB server, I opened MongoDB Compass and clicked on the "New Connection" button. I then clicked the "Create Database" button to create a new one after establishing a link.



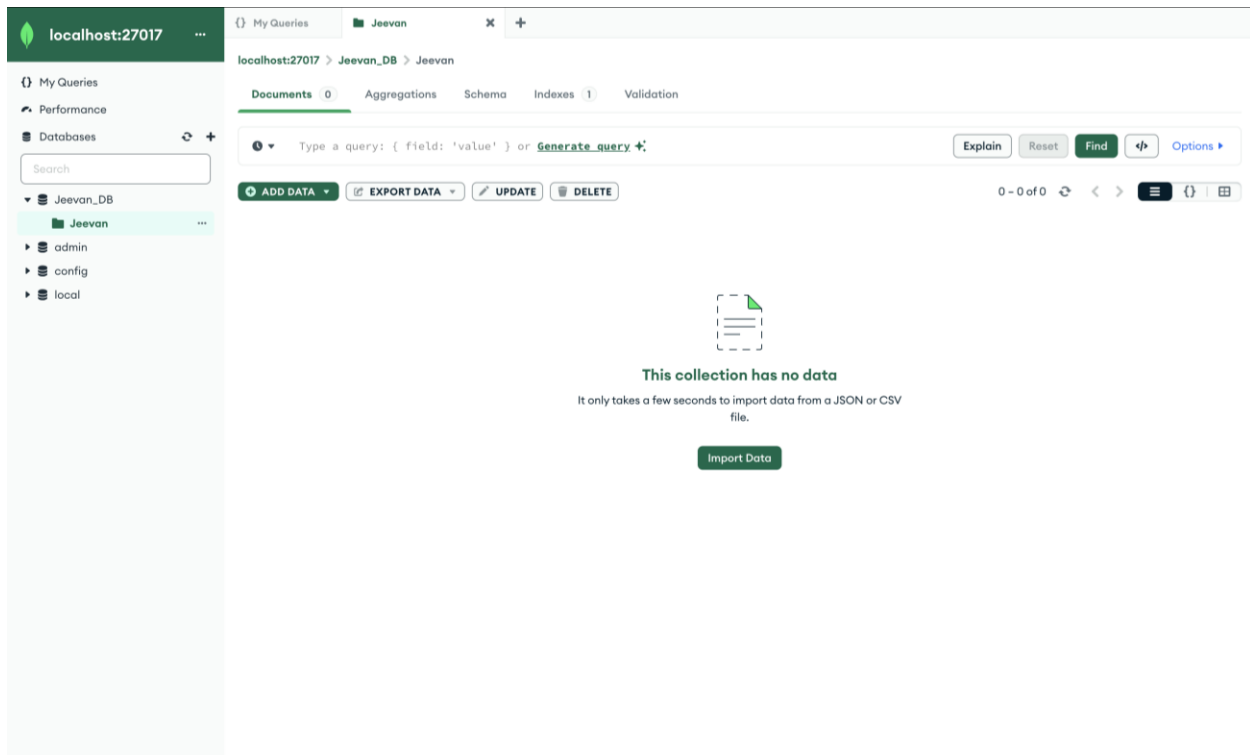
Then by using the connect button we must connect the mongo DB with local host, and we must make sure of our mongo dB in running state in settings.



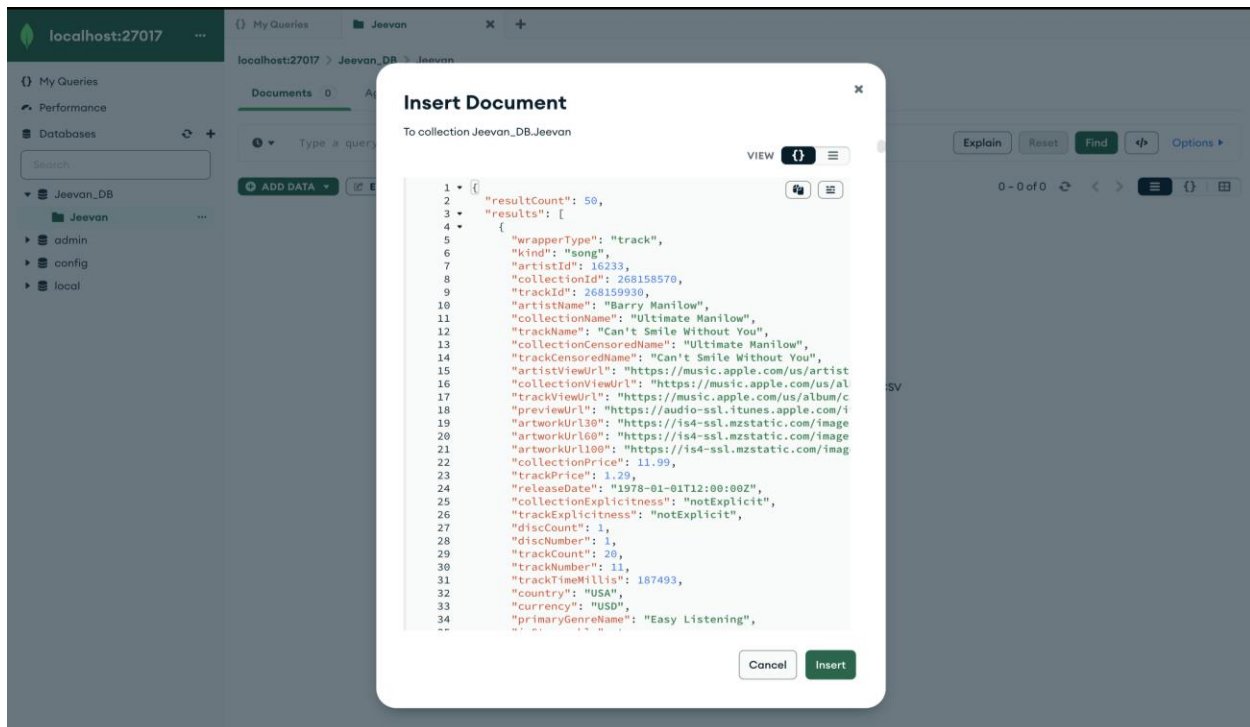
Then we will be creating the new database title and collection name as we need by using + beside databases, in my case I have given Jeevan_DB and Jeevan respectively.



After we are done with that, the template should look like this.



By using add data from above, we can either add the data directly by using csv files or we can even paste the code from the data as below.



We must add more files like this to perform the operations as below.

The screenshot shows the MongoDB Compass interface. On the left sidebar, the 'Jeevan' database is selected. The main panel displays the 'Jeevan' collection with a list of documents. The documents are as follows:

- Document 1: `{ "_id": ObjectId("66a543576408ab96cf07c013"), "resultCount": 50, "results": Array(50) }`
- Document 2: `{ "_id": ObjectId("66a543c46408ab96cf07c015"), "ProductCatalog": Array(8) }`
- Document 3 (Selected): `{ "_id": ObjectId("66a543d56408ab96cf07c017"), "product1": Array(8) }`
- Document 4: `{ "_id": ObjectId("66007628d7d573769febee55"), "name": "standard desk chair", "price": 150, "brand": "CheapCo", "type": "chair" }`
- Document 5: `{ "_id": ObjectId("6600777fd7d573769febee56"), "name": "basic desk chair", "price": 100, "brand": "Roughrider", "type": "chair" }`
- Document 6: `{ "_id": ObjectId("6600795ed7d573769febee5f"), "name": "standard with drawer", "price": 300, "brand": "Officedepo", "type": "table" }`

The selected document (Document 3) is highlighted, and its details are shown in the right pane. The details include the document's ID, the 'product1' array, and a trashcan icon for deletion.

To amend or get a product up to date start by searching for the Patient's file within the collection and choose it. Next make all necessary changes then be sure to hit “Save” so that those changes would take effect. To delete a patient’s document, find that specific one press delete button represented by trashcan icon next to it confirm deletion process afterward.

localhost:27017 > Jeevan_DB > Jeevan

Documents 6 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

ADD DATA EXPORT DATA UPDATE DELETE

1 - 6 of 6

```

{
  "_id": ObjectId("66a543d56408ab96cf07c017"),
  "product1": Array (8)
  * 0: Object
    * PutRequest: Object
      * Item: Object
        * Id: Object
          N: "101"
        * Title: Object
          S: "Book 101 Title"
        * ISBN: Object
          S: "111-1111111111"
        * Authors: Object
          * L: Array (1)
            * 0: Object
              S: "Author1"
        * Price: Object
          N: "2"
        * Dimensions: Object
          S: "8.5 x 11.0 x 0.5"
        * PageCount: Object
        * InPublication: Object
      * ProductCategory: Object
        S: "Book"
    * 1: Object
      * PutRequest: Object
        * Item: Object
          * Id: Object
            N: "102"
          * Title: Object
            S: "Book 102 Title"
          * ISBN: Object
            S: "222-2222222222"
          * Authors: Object
            * L: Array (2)
              * 0: Object
                S: "Author1"

```

Change the Information in your document: - Find documents you want to change from patients' collection. - Click on them to open - Change what is necessary. - Click on the "Update" button so that changes can be saved.

localhost:27017 > Jeevan_DB > Jeevan

Documents 6 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

ADD DATA EXPORT DATA UPDATE DELETE

1 - 6 of 6

Clone document

```

{
  "_id": ObjectId("66a543d56408ab96cf07c013"),
  "resultCount": 50
  * results: Array (50)
}

{
  "_id": ObjectId("66a543c46408ab96cf07c015"),
  "ProductCatalog": Array (8)
}

{
  "_id": ObjectId("66a543d56408ab96cf07c017"),
  "product1": Array (8)
}

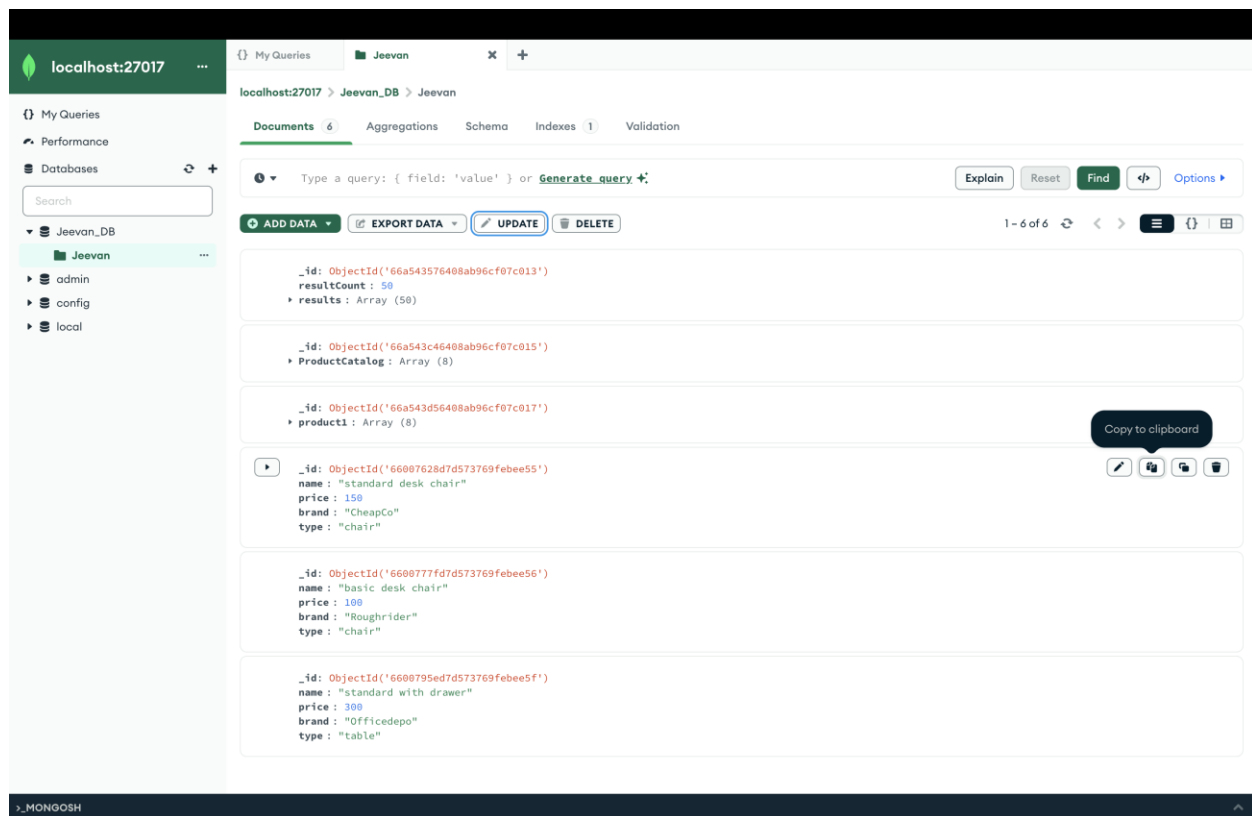
{
  "_id": ObjectId("66007628d7d573769febee55"),
  "name": "standard desk chair",
  "price": 150,
  "brand": "CheapCo",
  "type": "chair"
}

{
  "_id": ObjectId("6600777fd7d573769febee56"),
  "name": "basic desk chair",
  "price": 100,
  "brand": "Roughrider",
  "type": "chair"
}

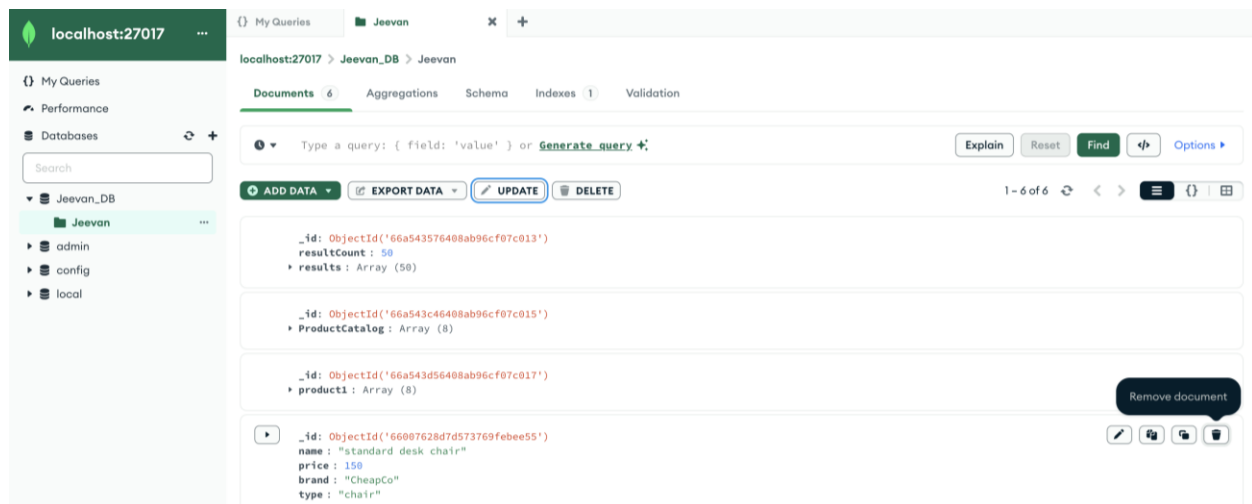
{
  "_id": ObjectId("6600795ed7d573769febee5f"),
  "name": "standard with drawer",
  "price": 300,
  "brand": "Officedepo",
  "type": "table"
}

```

By using the clone button, we can clone the entire document just like that.



We can even copy an entire document by using the copy button and it will copy the full json file.



Just by using the Remove document button we can even remove the entire file.

Performing simple queries on the database.

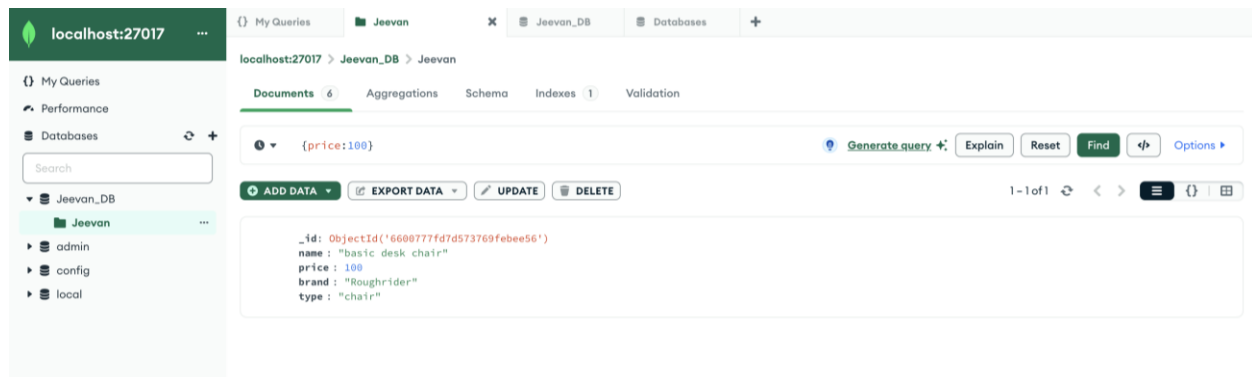
The screenshot shows the MongoDB Compass interface. On the left, the sidebar displays the connection 'localhost:27017' and a list of databases: 'Jeevan_DB' and 'Jeevan'. The 'Jeevan' database is selected, showing collections 'admin', 'config', and 'local'. The main panel shows the 'Jeevan' collection with a query filter of `{"type": "chair"}`. The query results are displayed in a table with two documents:

Document
<pre>{ "_id": ObjectId('66097628d7d573769febee55'), "name": "standard desk chair", "price": 150, "brand": "CheapCo", "type": "chair" }</pre>
<pre>{ "_id": ObjectId('6609777fd7d573769febee56'), "name": "basic desk chair", "price": 100, "brand": "Roughrider", "type": "chair" }</pre>

Finding the type by using char as its attribute.

The screenshot shows the MongoDB Compass interface. On the left, the sidebar displays the connection 'localhost:27017' and a list of databases: 'Jeevan_DB' and 'Jeevan'. The 'Jeevan' database is selected, showing collections 'admin', 'config', and 'local'. The main panel shows the 'Jeevan' collection with a query filter of `{"Gender": "Male"}`. The query results are displayed as 'No results' with the message 'Try modifying your query to get results.'

Next, we will be using the Gender as keyword and finding male, which resulted in zero because we don't have such type of attributes.



Next, we will be searching by using the price as key and "100" value.

Conclusion:

This lab aimed at discovering basic things like using MongoDB database through graphical user interface tools. The first thing we did was to install all the necessary software and ensure that it was working properly. The next thing was to connect to a local instance of MongoDB using the said interface. After connecting, we moved on to creating a fresh database along with a collection in it thereby is to state names for both. After setting up the database, we concentrated on inserting new entries known as documents in MongoDB.

This lab aimed at exploring basics such as using MongoDB database through GUIs. First, we installed all the required software available and ensured they functioned well. Next thing was connecting local instances to the interface which is used for Mongo. So, after connecting we proceeded to create a fresh database as well as housing it one collection; thus, speaking names for both were specified.

Then we turned our attention towards adding new inputs denoted as documents in MongoDB considering that there comes out with certain format into which they should come forth using interpenetrated man's hands symbolizing their entering manually linearly.

Thereafter, practical examples were demonstrated through the addition of several documents possessing different sets of figures. Multiple actions were performed upon insertion over various items grading these documents again.

These acts included such operations as altering data present inside document or cloning similar items eliminating from predecessor collection respectively whereupon every of these helped us learn about managing information properly stored in data repositories.